

JSON Parsers for JSONTestSuite: Candidate Additions

Compiled from 20 sources and ranked by likelihood of deviating from RFC 8259

1. Scope and method

The previous version of this document came only from my own knowledge. This version is built from published implementation lists, benchmark repositories, security and academic surveys, and documentation pages (Appendix B). Roughly 400 named libraries and tool families appear below. Anything that was already in your repository listing, or in the Prodigysec fork README, is excluded from the candidates and collected in Appendix A.

This is the most complete list I could assemble from those sources, but it is not exhaustive. JSON implementations number in the thousands: a single npm search for JSON stream parsers reported more than 1,000 packages. To go further, query the package registries directly (npm, PyPI, crates.io, Maven Central, NuGet, pkg.go.dev, RubyGems, Packagist, Hackage, Hex, LuaRocks, CPAN) and the GitHub topic pages for json-parser.

How entries are ranked

Each entry carries a basis tag so you can see how much weight it deserves:

- **[Measured]** someone ran the library against a conformance suite or documented specific failures. Strongest evidence.
- **[Documented]** the library's own documentation, source or a survey states lenient behaviour, a dialect, or a claim of compliance.
- **[Heuristic]** my inference from design (hand-written, minimal, non-validating, niche) or from lineage. No test data found. Treat as a hypothesis to test.

The Sources column uses S1 to S20 (Appendix B). "M" marks claims that come from my own earlier knowledge and were not independently confirmed.

"Deviation" means accepting input RFC 8259 forbids (n_ fixtures) or rejecting valid input (y_ fixtures). Duplicate-key policy, huge numbers, lone surrogates and nesting limits are implementation-defined (i_ fixtures) and are not counted as violations.

2. Measured evidence found

Three sources ran comparable suites, and they anchor the ranking. Counts below are copied from the repositories; I did not rerun them.

Python libraries against the JSON minefield (TkTech/json_benchmark, regenerated 2026-07-13)

Library	Expected result	Invalid input accepted	Valid input rejected	Implementation-defined accepted
yyjson	283	0	0	6
orjson	283	0	0	6

Library	Expected result	Invalid input accepted	Valid input rejected	Implementation-defined accepted
msgspec	283	0	0	6
simdjson	283	0	0	4
ijson	282	1	0	16
python-rapidjson	277	6	0	6
ujson	267	16	0	21

Go libraries against the RFC 8259 test suite (go-json-experiment/jsonbench)

Library	String	Number	Object	Array	Other
go-json-experiment/json (v2)	0	0	0	0	0
encoding/json (v1)	10	0	0	0	0
segmentio/encoding	10	0	0	0	0
sugawarayuuta/sonnet	10	0	0	0	0
json-iterator/go	10	4	0	0	0
bytedance/sonic	28	0	0	1	0
goccy/go-json	30	52	20	17	10

Failures counted per category. Most of the encoding/json-style string failures concern UTF-8 validation, which the benchmark author treats as incorrect per RFC 8259 section 8.1.

Lua libraries against wjson's adversarial suite

Library	Valid inputs rejected	Malformed Unicode rejected	Other malformed rejected
dkjson (already in repo)	0/26	0/17	0/32
rxl/json.lua	0/26	12/17	32/32
lunajson	0/26	17/17	32/32
lua-cjson	0/26	17/17	31/32

The wjson excerpt does not label its columns. I read the first as valid inputs wrongly rejected and the other two as malformed inputs correctly rejected; check the repository before quoting this.

Bishop Fox survey: behaviours outside the norm (49 parsers)

- Accepts comments although it is not a JSON5/HJSON parser: Ruby JSON (ext and pure), Oj, Yajl, Go jsonparser, Go gojay, Java Gson, Genson and fastjson, C# Newtonsoft.Json, Utf8Json, Jayrock.Json and Manatee.Json.
- Character truncation or key collisions: Python ujson (unpaired surrogates), PHP and Rust json5, Ruby simdjson (stray quotes), C# Jayrock, Ruby stdlib, JavaScript json5 (stray backslash).
- Huge numbers converted to Infinity, strings, 0 or rounded: Python json and ujson, Java Jackson, Genson and Jodd, C# Manatee, Newtonsoft and Utf8Json, Ruby Oj, PHP jsonlint, Go jsonparser.
- First-key duplicate precedence: Go jsonparser and gojay, C++ RapidJSON, Java json-iterator, Elixir Jason and Poison, Erlang jsone. (Allowed by the RFC but a risk for interoperability.)
- Two unnamed parsers crashed with segmentation faults on malformed input (names withheld by the author).

Several of these (Gson, Jackson, Newtonsoft, Oj, Yajl, Ruby stdlib, Python ujson) are already in your repo, which is a useful cross-check: independent evidence says they should show deviations when you run them.

3. Tier 1: very likely to deviate

Measured failures, documented leniency, or dialects that deviate by design.

1A. Measured deviations

Parser / library	Language	Basis and notes	Sources
goccy/go-json	Go	[Measured] Against the RFC 8259 suite: 30 string, 52 number, 20 object, 17 array and 10 other failures. Also crashes and memory-corruption bugs in round-trip tests.	S6
bytedance/sonic	Go	[Measured] 28 string failures and 1 array failure. Does not validate UTF-8 on marshal or unmarshal.	S6
json-iterator/go	Go	[Measured] 10 string and 4 number failures. Ignores invalid UTF-8 when unmarshaling and accepts some invalid numbers.	S6
buger/jsonparser	Go	[Measured] Does not validate the document. First-key precedence, accepts comments, and turns huge numbers into 0 without an error.	S5
francoispqt/gojay	Go	[Measured] First-key precedence for duplicate keys and accepts comments.	S5
json-stream	Python	[Measured] Issue report: passes all positive cases but accepts malformed inputs such as [3[4]] and [, ""].	S7
python-rapidjson	Python	[Measured] Accepted 6 inputs the suite says must be rejected (plus 6 implementation-defined accepts).	S7
ijson	Python	[Measured] Accepted 1 input that must be rejected; accepted 16 implementation-defined inputs.	S7
Fastjson 1.x	Java	[Measured] Version 1.2.73 accepts comments (fastjson2 is already in the repo; 1.x is a different codebase).	S5,S4
Genson	Java	[Measured] Accepts comments and emits Infinity for 1.0e4096.	S5,S4
json-iterator (jsoniter)	Java	[Measured] First-key precedence, comment truncation and inconsistent precedence between get and serialize.	S5,S4
Utf8Json	C#	[Measured] Accepts comments and rounds very large numbers to a different value.	S5,S9
Jayrock.Json	C#	[Measured] Accepts comments and treats a stray backslash in a key as a collision.	S5
Manatee.Json	C#	[Measured] Accepts comments and converts huge exponents to Infinity.	S5,S1
JSON pure variant	Ruby	[Measured] The pure-Ruby stdlib parser accepts comments and has stray-backslash key collisions (the ext variant behaves similarly).	S5
simdjson gem (simdjson_ruby)	Ruby	[Measured] Version 1.0.1 showed stray-quote key collisions and comment truncation, even though it wraps simdjson.	S5,S3
rxj/json.lua	Lua	[Measured] In wjson's adversarial suite it rejected only 12 of 17 malformed Unicode inputs (other categories were rejected in full).	S11

Parser / library	Language	Basis and notes	Sources
libjson (SourceForge)	C++	[Measured] Could not parse UTF-8 strings when integrated into nativejson-benchmark.	S2
Boost.PropertyTree	C++	[Measured] Converts numbers, true, false and null to strings. Not a true JSON data model.	S2
pjson, mikeando/FastJson, sheredom/json.h	C / C++	[Documented] Do not parse numbers themselves (the benchmark post-processes them), so number grammar such as leading zeros is unchecked by the parser.	S2

1B. Documented lenient modes or lenient by design

Parser / library	Language	Basis and notes	Sources
json-smart	Java	[Documented] Default mode is MODE_PERMISSIVE, described in its source as accepting lots of non-standard JSON. Strict modes (MODE_RFC4627, MODE_STRICTEST) exist but are opt-in.	S18
Groovy JsonSlurper (LAX / JsonParserLax)	Groovy	[Documented] Lax parser accepts comments, single quotes and unquoted strings.	S18
jsonic (Go and TypeScript)	Go / TS	[Documented] Relaxed-JSON grammar: unquoted keys, implicit objects, comments, trailing commas, single quotes.	S18
jsonvx	Go	[Documented] Configurable parser with relaxed options (hex numbers, single quotes, unquoted keys, trailing commas, comments, NaN/Infinity).	S18
SmarterJSON	Ruby	[Documented] Accepts the whole human-JSON superset with no strict mode: comments, trailing commas, unquoted keys, NaN/Infinity, Python literals.	S14
SafeJsonParser, best-effort-json-parser	TS / JS	[Documented] Forgiving parsers for LLM-style output: comments, trailing commas, single quotes, markdown fences, partial documents.	S18,S10
UltraJSON lineage: ujson4c, go-ujson, nujson, ujson (C++)	C / Go / Python / C++	[Heuristic] Same family as Python ujson, which accepted 16 invalid inputs in the Python benchmark and showed surrogate-truncation collisions. ujson itself is already in the repo.	S1,S2,S6,S7,S5
Configuru	C++	[Heuristic] Configuration-oriented parser; from memory it allows comments and trailing commas. Verify before relying on this.	S2,M

1C. Superset dialects (deviate by design)

Parser / library	Language	Basis and notes	Sources
JSON5 (json5, reference implementation)	JavaScript	[Documented] Superset by design: comments, single quotes, unquoted keys, trailing commas, hex numbers. Bishop Fox also found stray-backslash key collisions.	S5,S10
json5 crate; serde-hjson	Rust	[Documented] JSON5 and HJSON dialects. json5 0.3.0 showed character-truncation key collisions.	S5
colinodell/json5; SerafimArts/Json5; hiroto-k/JSON5-php	PHP	[Documented] JSON5 parsers. colinodell/json5 tries native json_decode first and falls back to JSON5.	S15,S5
json5 gem	Ruby	[Documented] JSON5 dialect (v0.0.1 in the survey).	S5
json5 (PyPI)	Python	[Heuristic] JSON5 dialect.	M

Parser / library	Language	Basis and notes	Sources
HJSON implementations; JSON6	Multiple	[Documented] Relaxed dialects (HJSON is a JSON5-like design; JSON6 is JSON for humans, ES6).	S5,S19
jsonc-parser; stream-json (JSONC mode)	JavaScript	[Documented] JSONC: JSON with comments (stream-json ships a JSONC parser and verifier).	S10,M
Amazon Ion (text form)	Multiple	[Heuristic] Superset with comments, symbols and timestamps; useful as a contrast case.	M

4. Tier 2: moderately likely, or unknown

No conformance data found, so the order inside this tier reflects design risk. 2A is most likely to hide gaps; 2C is the unrated long tail.

2A. Minimal, embedded or hand-rolled parsers

Parser / library	Language	Basis and notes	Sources
mu_json, LibU json, M's JSON parser (SourceForge mjson), lwjson, cisson, nanoJSONc, jsonsl, WJElement	C	[Heuristic] Small hand-written C parsers listed on json.org; footprint-oriented, no independent conformance results found.	S1
microjson (esr), mjson (cpq), frozen (cesanta), tiny-json, cson, nosjob, mm_json.h	C	[Heuristic] Minimal embedded-style parsers. Number and escape validation is the usual weak spot.	S1,S2,M
gason (vjson successor)	C++	[Heuristic] Speed-oriented parser in the C++ benchmark; no conformance results in the sources I read.	S2
qajson4c, facil.io, ULib, StiX Json, last.json, vincenthz/libjson, NXJSON	C / C++	[Heuristic] Lesser-known parsers included or attempted in nativejson-benchmark; verify with the suite.	S2
JsonBox, CAJUN, json-voorhees, jvar, jeayeson, Jzon	C++	[Heuristic] Small C++ libraries from the benchmark list.	S2
nbsdx/SimpleJSON, MJPA/SimpleJSON, hjiang/jsonxx, tunnuz/JSON++, minijson, jsonme--, jsOVon	C++	[Heuristic] Hobby-scale single-header style libraries.	S1,S2
qmjson, JsonWax (Qt), JSON library for IoT (Real Time Logic), Qentem-Engine, QJson	C++	[Heuristic] Qt and embedded JSON libraries from json.org.	S1
univalue	C++	[Heuristic] Bitcoin Core's JSON library; narrow, purpose-built.	S1

2B. Performance-oriented, feature-rich and ecosystem libraries

Parser / library	Language	Basis and notes	Sources
parson	C	[Heuristic] Widely used C parser; not in the repo and no conformance data found.	S1
ArduinoJson	C++	[Heuristic] Embedded-focused; tolerant by design in places.	S1,S2

Parser / library	Language	Basis and notes	Sources
JsonCpp	C++	[Heuristic] Has lenient options (comments, NaN) that may be on by default in some builds.	S1,S2,M
dropbox/json11, Poco::JSON, Folly dynamic, Qt QJsonDocument, C++ REST SDK (Casablanca), ThorsSerializer	C++	[Heuristic] Library-bundled parsers with differing validation; benchmarked in nativejson-benchmark.	S2,S1
Boon	Java	[Heuristic] Groovy's lax parser is Boon-derived (from memory). A recent benchmark run reports Boon produced no scores on JDK 25 due to an initialization failure.	S4,S18,M
json-io; Jettison; XStream (JSON driver); Jodd JSON	Java	[Documented] A study of 20 Java libraries found broad behavioural diversity on ill-formed input. Jodd also prints +Infinity for 1.0e4096.	S17,S5,S4
Moshi; Jackson jr; kotlinx.serialization; Klaxon	Java / Kotlin	[Documented] Strict by default; kotlinx.serialization isLenient is opt-in and allows unquoted keys and strings.	S16,S4
DSL-JSON; avaje-jsonb; minimal-json; mjson (bolero); underscore-java; antonsjava/json; Tapestry JSON; LoganSquare; purejson; qson; Quickbuf JSON	Java	[Heuristic] Performance-oriented or niche libraries from the Java benchmark family.	S4
Flexjson; json-lib; JSON-util; Argo; jsonij; jjson; FOSS Nova JSON; Corn CONVERTER; cookjson; Stringtree; SOJO; json-taglib; esson; JSONUtil; MOXy	Java	[Documented] json.org Java entries; several were part of the 20-library behavioural study.	S1,S17
Play JSON; spray-json; uPickle; weePickle; json4s; Argonaut; ZIO JSON; jsoniter-scala; borer; smithy4s-json; zio-schema- json; Lift JSON; scalajack; tupson; AVSystem scala- commons	Scala	[Heuristic] Many delegate to Jackson or Jawn; jsoniter-scala, uPickle, ZIO JSON and borer have their own readers.	S16
Cheshire; jsonista; charred	Clojure	[Heuristic] Cheshire and jsonista wrap Jackson; charred is a custom reader.	M
Jil; NetJSON; SpanJson; ServiceStack.Text; fastJSON; FastJsonParser; LightJson	C#	[Heuristic] Performance-first serializers benchmarked in several .NET comparisons; no suite results found.	S9,S1
JSON for .NET (csjson); Liersch.Json; Liersch.JsonSerialization; JSON Essentials; JSON_checker (C#)	C#	[Heuristic] json.org C# entries.	S1
LitJson; JavaScriptSerializer; DataContractJsonSerializer; Windows.Data.Json; FSharp.Data; Thoth.Json; Swifter.Json	C# / F#	[Heuristic] Legacy or niche .NET parsers from my own knowledge.	M

Parser / library	Language	Basis and notes	Sources
segmentio/encoding	Go	[Measured] 10 string failures, all tied to UTF-8 validation (same as encoding/json).	S6
sugawarayuuta/sonnet	Go	[Measured] 10 string failures (UTF-8 related).	S6
mailru/easyjson; pquerna/ffjson; valyala/fastjson; bitly/go-simplejson; jscan; ojg	Go	[Heuristic] Code-generated or speed-oriented Go libraries; go-simplejson delegates to encoding/json.	S6,S19,M
simd-json (Rust port); simdjson-rust	Rust	[Heuristic] Ports and wrappers of simdjson; ports can diverge from the original.	S3,S8
sonic-rs; jiter	Rust	[Heuristic] SIMD parsers. sonic-rs defers decoding of strings and offers unchecked get variants that skip validation.	S8,M
Pikkr; A-JSON; GJSON (Rust); tinyjson; nanoserde	Rust	[Heuristic] Specialised or lightweight Rust parsers from benchmark repos.	S8
asmjson; kowito-json; rust- json-parse	Rust	[Heuristic] Experimental high-speed parsers; the authors recommend serde_json or sonic-rs for production.	S8
yapic.json; hyperjson; pydantic-core	Python	[Heuristic] Native-code parsers from Python comparisons.	S7,M
json2.js; json_parse.js (Crockford)	JavaScript	[Heuristic] Original polyfills; the json2 variant validates with a regex before eval.	S1,S19
JSON 3	JavaScript	[Heuristic] Polyfill listed in awesome-json.	S19
clarinet; Oboe.js; jsonparse; json-stream (npm); big-json; yieldable-json	JavaScript	[Heuristic] Streaming parsers with relaxed state machines; clarinet is a SAX-style parser forked from sax-js.	S1,S10
stream-json	JavaScript	[Documented] Streaming SAX-style tokenizer with JSON, JSONL and JSONC parsers.	S10
parse-json; json-parse- better-errors; jsonlint; json- buffer; buffer-json	JavaScript	[Documented] Surveyed or listed; Bishop Fox reported no uncommon behaviour for json-buffer, buffer-json and parse-json.	S5,S10
seld/jsonlint; halaxa/json- machine; salsify/jsonstreamingparser; Webmozart JSON	PHP	[Documented] Surveyed by Bishop Fox; jsonlint rounded 96 nines to 9223372036854775807.	S5,S19
jsond	PHP	[Heuristic] Basis of PHP's native parser; included for completeness.	M
rapidjson-ruby; fast_jsonparser; json-stream (Ruby)	Ruby	[Heuristic] Bindings or streaming parsers; fast_jsonparser documents known incompatibilities with stdlib JSON.	S14,S1
jsone	Erlang	[Measured] First-key precedence for duplicate keys (allowed by the RFC but unusual).	S5,S12
jsx; jiffy	Erlang	[Documented] Surveyed by Bishop Fox with no uncommon behaviour reported.	S5,S12
erl-json (exograd); jhn_stdlib json; ojson	Erlang	[Heuristic] Less common Erlang parsers.	S1,S12
Exneus; torque (sonic-rs NIF); glazer	Elixir / Erlang	[Heuristic] Wrappers and NIF bindings around OTP json, sonic-rs and custom code.	S12

Parser / library	Language	Basis and notes	Sources
IkigaJSON; swift-json (rarestype); SwiftyJSON; HandyJSON; Himotoki; JBird; universal; SmartCodable; CodableJSON	Swift	[Heuristic] IkigaJSON and swift-json are custom parsers; SwiftyJSON and others sit on Foundation (inherited behaviour).	S13,M
Mantle; JSONModel; yajl-objc	Obj-C	[Heuristic] Model mappers on NSJSONSerialization, plus a YAJL binding.	S1,S13,M
Jay; Argo; Unbox	Swift	[Heuristic] Older Swift parsers/mappers.	M
lua-yajl; lua-cjson	Lua	[Measured] lua-cjson rejected 17/17 and 31/32 malformed inputs in wjson's suite, so it sits near the strict end. lua-yajl wraps YAJL.	S11
JSON::SIMD	Perl	[Heuristic] Full JSON module that uses simdjson for decoding.	S3
JSON::MaybeXS; JSON::Create; JSON::Streaming::Reader	Perl	[Heuristic] MaybeXS is a front end that picks a backend, so behaviour depends on the backend.	M
Racket json-parsing; MZScheme JSON; JSON-struct; Guile json; Chicken json	Scheme / Racket	[Heuristic] Compact pure-language implementations.	S1,M
cl-json; jsown; yason; jonathan; shasht	Common Lisp	[Heuristic] Each has its own tokenizer.	S1,M
JSON.jl; JSON3.jl	Julia	[Heuristic] Julia parsers from memory.	M
std.json; asdf; vibe.data.json; mir-ion	D	[Heuristic] std.json is the standard library; asdf and vibe.data.json are third-party; mir-ion parses several formats.	S1,M
jsonm; ezjsonm; jsonaf	OCaml	[Heuristic] jsonm is a streaming codec listed on json.org.	S1,M
json (Text.JSON); RJson; hjson; hermes-json	Haskell	[Heuristic] hermes-json binds simdjson; hjson is a dialect.	S1,S3,M
jsony; gemmaJSON	Nim	[Heuristic] gemmaJSON is based on simdjson bindings.	S3,M
JSON-Simd; JSON::Fast	Raku	[Heuristic] JSON-Simd binds simdjson.	S3,M
RJSONIO; yyjsonr; rcppsimdjson	R	[Heuristic] rcppsimdjson binds simdjson; RJSONIO is an older parser.	S3,M
dart:convert (JsonCodec)	Dart	[Heuristic] Standard library parser listed on json.org.	S1,M

2C. Long tail from json.org and awesome-json

Parser / library	Language	Basis and notes	Sources
8th json; ActionScript3 (as3corelib); GNATCOLL.JSON (Ada); JSON-ADVPL; APL $\square$ JSON	Various	[Heuristic] json.org entries for niche languages. No conformance data found.	S1
JSON for ASP; JSON ASP utility class; VB-JSON; PW.JSON; .NET-JSON-Transformer	ASP / Visual Basic	[Heuristic] Classic ASP and VB libraries.	S1
rhawk json.awk	AWK	[Heuristic] AWK implementation (JSON.awk itself is already in the repo).	S1
bmx-rjson	BlitzMax	[Heuristic] Single-purpose library.	S1

Parser / library	Language	Basis and notes	Sources
Redvers COBOL JSON Interface; GnuCOBOL JSON	COBOL	[Heuristic] COBOL implementations.	S1,M
SerializeJSON	ColdFusion	[Heuristic] Serializer; parsing via the platform.	S1
Delphi Web Utils; JSON Delphi Library; SuperObject; JsonTools; System.JSON	Delphi / Pascal	[Heuristic] Delphi libraries (Free Pascal fcl-json is already in the repo).	S1,M
JSON in TermL	E	[Heuristic] Embedding in TermL.	S1
Fantom Json; FileMaker JSON; LabVIEW flatten; LiveCode mergJSON; LotusScript JSON LS; M DataBallet; Net.Data netdata-json	Various	[Heuristic] Single-platform libraries from json.org.	S1
json-fortran; YAJL-Fort; jsonff	Fortran	[Heuristic] Fortran parsers (YAJL-Fort wraps YAJL).	S1
groovy-io	Groovy	[Heuristic] From the json-io author.	S1
JSONlab; MATLAB File Exchange 20565 and 23393	MATLAB	[Heuristic] MATLAB implementations.	S1
PascalScript JsonParser; Photoshop JSON scripting; picolisp-json; Public.Parser.JSON and JSON2 (Pike); pljson and PL/JSON; PureBasic JSON; PuRestJson (Pure Data); json.r (Rebol); RPG JSON Utilities; Squeak; Tcl JSON (wiki); fwJSON and vfpjson (Visual FoxPro); Wing json-type	Various	[Heuristic] Long-tail entries from json.org and awesome-json.	S1,S19
Jshon; JSON.sh derivatives; jwalk; jsawk; json (CLI); jl; fx	Shell / CLI	[Heuristic] Shell and command-line tools; JSON.sh is already in the repo.	S1,S19,M
gojq; jaq; jq; dasel; Miller; yq	CLI	[Heuristic] jq-like tools; gojq and jaq intentionally mirror jq, dasel/Miller/yq use other parsers.	M
PowerShell ConvertFrom-Json	PowerShell	[Heuristic] Backend differs by version (JavaScriptSerializer in 5.1, Newtonsoft in 7+), from memory.	M
MySQL/MariaDB JSON; Oracle JSON; SQL Server OPENJSON; DuckDB; ClickHouse; MongoDB Extended JSON	Databases	[Heuristic] Database-specific validation and extensions (SQLite JSON1 and PostgreSQL JSONB are already in the repo).	M

5. Tier 3: unlikely to deviate (controls and wrappers)

Strict by default or measured clean, plus wrappers that inherit another parser's behaviour. Add one representative of each family as a control rather than every wrapper.

Parser / library	Language	Basis and notes	Sources
yyjson	C	[Measured] 283 expected results, 0 accepted-invalid, 0 rejected-valid in the Python suite run (extensions are opt-in flags).	S7
orjson	Python / Rust	[Measured] 283 expected results, 0 deviations.	S7
msgspec	Python	[Measured] 283 expected results, 0 deviations.	S7
pysimdjson; cysimdjson; libpy_simdjson	Python	[Measured] simdjson itself had 0 deviations; these are bindings that inherit it.	S7,S3
go-json-experiment/json (JSONv2)	Go	[Measured] Zero failures against the RFC 8259 suite and the stricter RFC 7493.	S6
Euneus; Thoas; OTP json module	Erlang	[Documented] Claim full RFC 8259 conformance and were tested with JSONTestSuite (Euneus is already in the repo; OTP json is its base).	S12
@streamparser/json	JavaScript	[Documented] Claims to be fully compliant with the spec and JSON.parse.	S10
secure-json-parse	JavaScript	[Documented] Thin layer over JSON.parse with prototype-poisoning protection (surveyed, nothing unusual reported).	S5,S10
lunajson; wjson	Lua	[Measured] Rejected 17/17 and 32/32 malformed inputs (lunajson) in wjson's adversarial suite; wjson is built to be strict.	S11
Boost.JSON; jsoncons; taocpp/json; glaze	C++	[Heuristic] Modern, actively tested libraries; strict by default with extensions opt-in. Verify with the suite.	S2,M
Jakarta JSON-P (Glassfish); Eclipse Yasson; Apache Johnzon	Java	[Heuristic] Specification-driven implementations (Jakarta JSON-P / JSON-B).	S1,S4,M
Jawn; circe	Scala	[Heuristic] Jawn is a long-standing, well-tested parser that circe uses.	S16
trivet JSON parser; Kson; json-event-parser	Rust / Kotlin	[Documented] Each reports validating against JSONTestSuite (Kson is a superset with a documented edit list).	S20,M
simdjson ports: simdjson-go; simdjson-java; SimdJsonSharp	Go / Java / C#	[Heuristic] Reimplementations, so they can diverge, but they target full validation. Treat as 2B if you want to be cautious.	S3
simdjson bindings: simdjson_nodejs; simdjson_php; rcppsimdjson; simdjson_erlang; simdsone; lua-simdjson; hermes-json; JSON-Simd; JSON::SIMD; gemmaJSON; simdjson-dart; ZippyJSON; ReerJSON (yyjson)	Various	[Heuristic] Wrappers inherit simdjson (or yyjson for ReerJSON) behaviour, so one reference run covers them. The Ruby gem is the exception noted in Tier 1.	S3,S13
QuickJS; Duktape; Hermes; Rhino; Nashorn; GraalJS	JS engines	[Heuristic] ECMAScript JSON.parse implementations; V8, JavaScriptCore and Firefox are already in the repo.	M
Deno (V8); Bun (JavaScriptCore); SpiderMonkey standalone	JS runtimes	[Heuristic] Same engines as existing modes; low incremental value.	M
Elm Json.Decode; PureScript argonaut; Crystal; V; Odin; Gleam; Pony; Haxe	Mixed	[Heuristic] Wrap host-platform parsers or standard libraries, so behaviour is inherited.	M

6. Not directly applicable

- BSON, CBOR, MessagePack, UBJSON: binary formats; not testable with text fixtures (Bishop Fox also found duplicate-key issues in their parsers).
- JSON Schema validators (jsonschema, Ajv and similar), JSON Pointer and Patch libraries (json-patch-php, diffson), and query languages (JSONata): these consume an already parsed object, so they inherit the parser's behaviour.
- Serializer-only libraries (for example JSON-BUILD, which only builds output): no parser to test.

7. Suggested additions, in priority order

These combine strong evidence, real-world usage, and relevance to the number and leading-zero cases the repository is currently exercising.

- Highest value, measured failures: goccy/go-json, bytedance/sonic, json-iterator/go, buger/jsonparser, gojay.
- Documented leniency in widely used JVM and .NET libraries: json-smart (default permissive mode), Fastjson 1.x, Genson, json-iterator (Java), Groovy JsonSlurper, Utf8Json, Jayrock.Json, Manatee.Json.
- Scripting-language gaps: Ruby pure JSON and the simdjson gem, python-rapidjson, ijson, json-stream, rxi/json.lua, gjson.
- Dialect contrast cases: JSON5 (JavaScript), the json5 Rust crate, jsonc-parser, HJSON, SmarterJSON.
- Hand-rolled C and C++ parsers with no independent data: gason, microjson, mjson (cpq), frozen, tiny-json, lwjson, sheredom/json.h, pjson. These are the best bets for undiscovered leading-zero bugs.
- Strict controls (one each): yyjson, orjson or msgspec, go-json-experiment/json, Euneus or OTP json, lunajson or wjson, Boost.JSON or jsoncons.
- Skip: Deno, Bun and simdjson or yyjson wrappers (one reference run covers them), and any engine already represented by the V8, JavaScriptCore and Firefox modes.

8. Changes from the previous version

- Removed JSON::Tiny from the candidates: it is already in the repo (test_json_tiny.pl).
- Moved Boon, json-io, Jettison and similar Java libraries from Tier 1 to Tier 2B: I could only back them with memory or a general diversity study, not specific failures.
- Moved lua-cjson to the strict end: wjson's suite shows it rejecting almost every malformed input. I also dropped my earlier claim that it accepts NaN by default, which I could not confirm.
- Moved hand-written C parsers (gason, tiny-json, microjson and others) from Tier 1 to 2A: that ranking was a heuristic, and the tags now say so.
- Added measured evidence that was missing: Go (goccy/go-json, sonic, json-iterator), Python (python-rapidjson, ijson, json-stream), and Bishop Fox's comment-acceptance findings.
- Added lenient-by-design projects I had not listed: jsonic, jsonvx, SmarterJSON, SafeJsonParser, best-effort-json-parser.

Appendix A. Already covered by the repository

Taken from your pasted directory listing and the Prodigysec fork README. Excluded from the candidate tables. The README states the fork registers 94 parser configurations (versions and modes of the same library count separately) against a corpus of 327 parsing fixtures.

Group	Parsers and modes
C / C++	cJSON (1.7.3 and 2016-08), Jansson, json-c (two entries), JSON_checker (1 and 2, 2016-11), json-parser (udp), jsnm, RapidJSON, nlohmann/json, YAJL, picojson, sajson, JSON Spirit, pdjson, ccan/json, jsoncgv, amjson, libyaml, simdjson, QJson (Qt), JSONpp, opack
Java / JVM	Jackson, Gson, org.json, json-simple, nanojson 1.0 and 1.1, fastjson2, Actson, bfo, leastfixedpoint JSON, json-tree, mergebase Java2Json, opack, Clojure data.json, Scala Dijon
JavaScript / browsers	Node JSON.parse (strict UTF-8 mode), V8 native, JavaScriptCore, Chromium, Firefox, jq
Python	json (stdlib), simplejson, ujson, cjson, demjson
Ruby	json (stdlib), yajl-ruby, Oj (compat and strict), json_re
Perl	JSON::PP, JSON::XS, Cpanel JSON::XS, JSON::Parse, JSON::SL, JSON::Tiny, Mojo::JSON, Pegex::JSON, Marpa ESLIF ECMA-404
Swift / Obj-C	Apple JSONSerialization, Swift Foundation (Linux), Freddy (2), PMJSON (3), SBJson (4.0.3, 4.0.4, 5), STJSON, Swift-JSON 2017, JSONKit, TouchJSON
Other languages	Go encoding/json, serde_json, json-rust, rustc_serialize, PHP (several modes), Lua JSON and dkjson, Elixir (ExJSON, Jason, JSON, Poison), Erlang Euneus, Haskell aeson, OCaml Orsetto and Yojson, Nim, Zig, Crystal, Free Pascal fcl-json, R (jsonlite, rjson), Tcl rl_json, Bash JSON.sh, JSON.awk, Squeak JSON, .NET System.Text.Json and Newtonsoft, VHDL json
Databases	SQLite JSON1, PostgreSQL JSONB

Appendix B. Sources

Key	Source	Link
S1	json.org, Introducing JSON: implementation lists by language	https://www.json.org/json-en.html
S2	miloyip/nativejson-benchmark: 41 to 43 C/C++ libraries with conformance tests	https://github.com/miloyip/nativejson-benchmark
S3	simdjson: Bindings and Ports list	https://simdjson.org/software/
S4	fabienrenaud/java-json-benchmark and forks (Java and Kotlin libraries)	https://github.com/fabienrenaud/java-json-benchmark
S5	Bishop Fox, An Exploration of JSON Interoperability Vulnerabilities (49 parsers, 10 languages)	https://bishopfox.com/blog/json-interoperability-vulnerabilities
S6	go-json-experiment/jsonbench (Go correctness results) and other Go benchmark repos	https://github.com/go-json-experiment/jsonbench
S7	TkTech/json_benchmark (Python libraries run against the JSON minefield) and daggaz/json-stream issue 55	https://github.com/TkTech/json_benchmark
S8	Rust benchmarks and crate pages: rust-json-parsing-benchmarks, rust_serialization_benchmark, sonic-rs, lib.rs	https://github.com/AnnikaCodes/rust-json-parsing-benchmarks
S9	.NET benchmarks and READMEs: JsonBenchmark.Net, ngs-doo/json-benchmark, Utf8Json, Jil	https://github.com/AmirHJabari/JsonBenchmark.Net
S10	npm package pages and npm-compare (stream-json, @streamparser/json, jsonparse, clarinet, json5, others)	https://www.npmjs.com/package/stream-json

Key	Source	Link
S11	Lua: winterstream/wjson comparison, lunajson, lua-cjson performance notes	https://github.com/winterstream/wjson
S12	Erlang and Elixir: Exneus, Euneus, Thoas, simdjsone, glazer, forum posts	https://hexdocs.pm/exneus/readme.html
S13	Swift: IkigaJSON, ZippyJSON, ReerJSON, Swift Package Index	https://swiftpackageindex.com/keywords/json
S14	Ruby: Oj, rapidjson-ruby, fast_jsonparser, SmarterJSON	https://github.com/tilo/smarter_json
S15	PHP: simdjson_php, colinodell/json5, SerafimArts/Json5	https://github.com/colinodell/json5
S16	Scala and Kotlin: circe, REASY/scala-json-benchmark, jsoniter-scala, LibHunt, kotlinx.serialization docs	https://github.com/REASY/scala-json-benchmark
S17	Academic: Harrand et al., The Behavioral Diversity of Java JSON Libraries (20 libraries); Moller et al., Cross-Language Differential Testing of JSON Parsers (22 parsers)	https://arxiv.org/pdf/2104.14323
S18	Documentation of lenient parsers: json-smart JSONParser, Groovy JsonParserLax, jsonic, jsonvx	https://github.com/netplex/json-smart-v2/blob/master/json-smart/src/main/java/net/minidev/json/parser/JSONParser.java
S19	awesome-json lists (burningtree and frosencrantz)	https://github.com/burningtree/awesome-json
S20	Prodigysec/JSONTestSuite README and Seriot, Parsing JSON is a Minefield (what is already covered)	https://github.com/Prodigysec/JSONTestSuite
M	My own knowledge from the previous version of this document. Not independently verified; treat as leads, not findings.	n/a

Pages for the Prodigysec fork's parsers directory and Seriot's article could not be fetched in full (the directory page blocks automated access), so the already-covered list relies on your pasted listing plus the fork's README.